Mooketsi Vincent Magwaza

Full-stack engineer | Product systems, backend platforms, and operations

Gaborone, Botswana
mooketsimagwazajr@gmail.com
github.com/MooketsiMagwaza
mooketsimagwaza.github.io

PROFILE

Full-stack engineer who turns early product ideas into working systems: interface, API, data model, authentication, documentation, deployment path, and operational tooling. Strongest in backend, full-stack, and platform-focused work where product constraints and system boundaries matter.

CORE TOOLKIT

Backend	Rust, Axum, Python, FastAPI, Go, Java, REST APIs
Web	TypeScript, React, Next.js, Vite, responsive and accessible UI
Data	PostgreSQL, PostGIS, Redis, SQLAlchemy, sqlx, Alembic
Operations	Docker, nginx, GitHub Actions, OpenTelemetry, Prometheus, Grafana, Tempo
Native	Tauri, Rust, SwiftUI, Android platform APIs, Bluetooth HID

SELECTED PROJECT EXPERIENCE

Kori - offline-first wildlife sightings for Botswana

Next.js, TypeScript, MapLibre, IndexedDB | FastAPI, PostGIS, SQLAlchemy, Alembic | Docker and observability

Designed and built a four-app product for logging, reviewing, and sharing wildlife sightings in areas with unreliable connectivity.

- Built an offline field workflow with IndexedDB outbox storage, downloaded PMTiles packs, device-side photo metadata stripping, QR sharing, and routing constrained to known tracks.
- Produced a private Savuti pack from OpenStreetMap and Sentinel-2 data; verified satellite-first rendering, OSM-track guidance, attribution, and offline behavior in 16 of 16 automated checks.
- Implemented a FastAPI and PostGIS backend covering accounts, API keys, uploads, review, trips, publication delays, spatial privacy rules, migrations, and scheduled lifecycle jobs.
- Defined Docker Compose, OpenTelemetry, Prometheus, Grafana, Tempo, backup and recovery tooling, and CI checks. The backend is tested but not yet connected to the apps.

Tagwise - shared asset and stock register

React, TypeScript, Vite, MapLibre | Rust, Axum, PostgreSQL, Redis | nginx, Docker, OpenTelemetry

Built a responsive asset and stock workspace for organisations moving from spreadsheets to a shared register.

- Delivered register views, QR labels, browser scanning with manual fallback, Excel and CSV import validation, collaborative stock-takes, roles, conflict handling, and sites on a map.
- Shaped the application into a flat Apple-style interface with no decorative gradients while keeping the marketing page a distinct product presentation.
- Built five Rust services - identity, register, notifications, media, and ops - behind an nginx gateway with isolated PostgreSQL databases and Redis.
- Added database concurrency and idempotency tests, API docs, observability, Docker Compose, and GitHub Actions. The browser app still uses a labelled sample workspace and does not yet call the services.

SELECTED PROJECT EXPERIENCE - CONTINUED

Zenith - time tracking with intention

React, TypeScript, Vite, TanStack Router | Tauri 2 and Rust | SQLite, rustls, SPAKE2, Automerge

Created a local-first focus workspace built around one active card, one journal, and a clear record of deliberate work.

- Built timers, decks and cards, markdown journals, history, Zen mode, optional goals, progress, Pomodoro, calendar, and open-format import and export.
- Structured one repository for browser and Tauri desktop targets, an always-on-top pop-up, a static marketing site, and a Rust sync crate.
- Implemented an account-free device pairing and sync foundation using SPAKE2, mutual TLS with pinned certificates, Automerge, mDNS, and 36 local tests. The sync crate is not yet wired into the app.

University CS Docs - open learning platform

Next.js, MDX, reusable interactive components | CI, CodeQL, dependency review

Built and deployed a contribution-friendly study hub for University of Botswana computer-science courses.

- Organised course notes, code, quizzes, semester views, and a focus timer as reusable MDX-driven learning surfaces.
- Backed the repository with CI, CodeQL, dependency review, project governance, and a documented contributor path.

ENGINEERING PRACTICE

Product	Start from the user journey, then make system constraints visible in the interface and docs.
Architecture	Give data, services, permissions, and failure modes explicit owners; keep boundaries understandable.
Operations	Treat authentication, validation, rate limits, tracing, recovery, and deployment as product work.
Evidence	Separate verified behavior from planned work and sample data; record tests, screenshots, runbooks, and limits.

ADDITIONAL TECHNICAL WORK

Zenith Agent (Electron prototype), GlassHID (Android Bluetooth HID controller), and Obsidian Excalidraw Low Latency (pen-input companion). More project detail and current repository links are available in the portfolio.

Contact: mooketsimagwazajr@gmail.com | GitHub | Portfolio